DATABASE DOCUMENTATION

DevPilot AI

SQL Server Core ERD - 26 tables supporting the complete developer problem lifecycle

PROJECT	**DevPilot AI**
DOCUMENT VERSION	1.0
DOCUMENT TYPE	Database Documentation
DATABASE	SQL Server
DATABASE SCOPE	Core ERD - 26 core tables
ISSUE DATE	2 October 2026
PREPARED BY	Mahmoud Abou-Zaeid

Table of Contents

Document Overview 3
1. Database Design Principles 4
2. Table Catalog 4
3. Table Documentation 7
3.1 Users 7
3.2 Roles 7
3.3 UserRoles 8
3.4 EmailVerifications 8
3.5 Problems 9
3.6 ProblemRequests 10
3.7 ProblemAttachments 11
3.8 DebuggingAttempts 11
3.9 DeveloperFeedback 12
3.10 ProblemSolutions 12
3.11 KnowledgeArticles 13
3.12 ArticleVersions 13
3.13 Technologies 14
3.14 ProblemTechnologies 14
4. Practice / Challenge Tables 14
4.1 Challenges 14
4.2 ChallengeAttempts 15
4.3 ChallengeEvaluations 15
5. GitHub & Code Review Tables 16
5.1 GitHubAccounts 16
5.2 GitHubRepositories 16
5.3 CodeReviews 17
5.4 CodeReviewFindings 17
6. AI & Search Tables 18
6.1 AIProviders 18
6.2 AIModels 18
6.3 AIRequests 19
6.4 ProblemEmbeddings 20
6.5 SimilarityMatches 20
7. Relationship Summary 21
8. Core Business Data Flow 22
9. Token Optimization Data Model 22
10. Article Evolution Data Model 23
11. GitHub Data Flow 23

12. AI Data Flow 24
13. Important Constraints 24
13.1 Security 24
13.2 Privacy 24
13.3 History 24
13.4 Knowledge Quality 24
14. Indexing / Constraints to Define During Implementation 25
15. Transactional Areas 25
16. Core Tables vs Optional Supporting Tables 25
17. Database Implementation Order 26
18. Recent Approved Changes 26
19. Final Database Purpose 27

Document Overview

Property	Value
Document Version	1.0
Project	DevPilot AI
Database	SQL Server
Database Scope	Core ERD
Current Core Tables	26

Primary Purpose:

Support the complete DevPilot AI workflow:



The database also supports:

- Similar problem search
- Reuse of successful solutions
- LLM token/usage tracking
- Multiple AI providers and models
- GitHub integration
- Telegram knowledge lookup
- Continuous knowledge/article improvement

> **IMPORTANT**
> This document describes the current Core ERD. Supporting tables such as Notifications, DiscordEvents, and AuditLogs are intentionally excluded from the current Core ERD to keep the main database model focused and understandable.

1. Database Design Principles

No.	Principle	Description
1.1	Developer-Centered	The main business entity is the Developer (User).
1.2	Problem-Centered	A Problem is the central business object connecting debugging, solutions, knowledge articles, challenges, AI processing, and statistics.
1.3	Full Debugging History	The database keeps the requests, attempts, and developer feedback instead of storing only the final answer.
1.4	Knowledge Reuse	Successful solutions are stored so similar future problems can reuse them.
1.5	Knowledge Evolution	A problem can have multiple successful solutions and an article can have multiple versions.
1.6	GitHub Integration	GitHub accounts, repositories, challenges, and code reviews are stored and linked to developers.
1.7	AI Provider Independence	AI requests are linked to providers/models so the system can switch between multiple providers and track usage.

2. Table Catalog

The Core ERD contains 26 tables in six functional groups. Figure 1 on the following page shows every table and its key relationships.

IDENTITY	1. Users 2. Roles 3. UserRoles 4. EmailVerifications
PROBLEMS & DEBUGGING	5. Problems 6. ProblemRequests 7. ProblemAttachments 8. DebuggingAttempts 9. DeveloperFeedback 10. ProblemSolutions
KNOWLEDGE	11. KnowledgeArticles 12. ArticleVersions 13. Technologies 14. ProblemTechnologies
PRACTICE	15. Challenges 16. ChallengeAttempts 17. ChallengeEvaluations
GITHUB & CODE REVIEW	18. GitHubAccounts 19. GitHubRepositories 20. CodeReviews 21. CodeReviewFindings

AI & SEARCH	22. AIProviders 23. AIModels 24. AIRequests 25. ProblemEmbeddings 26. SimilarityMatches

Figure 1 -Core ERD

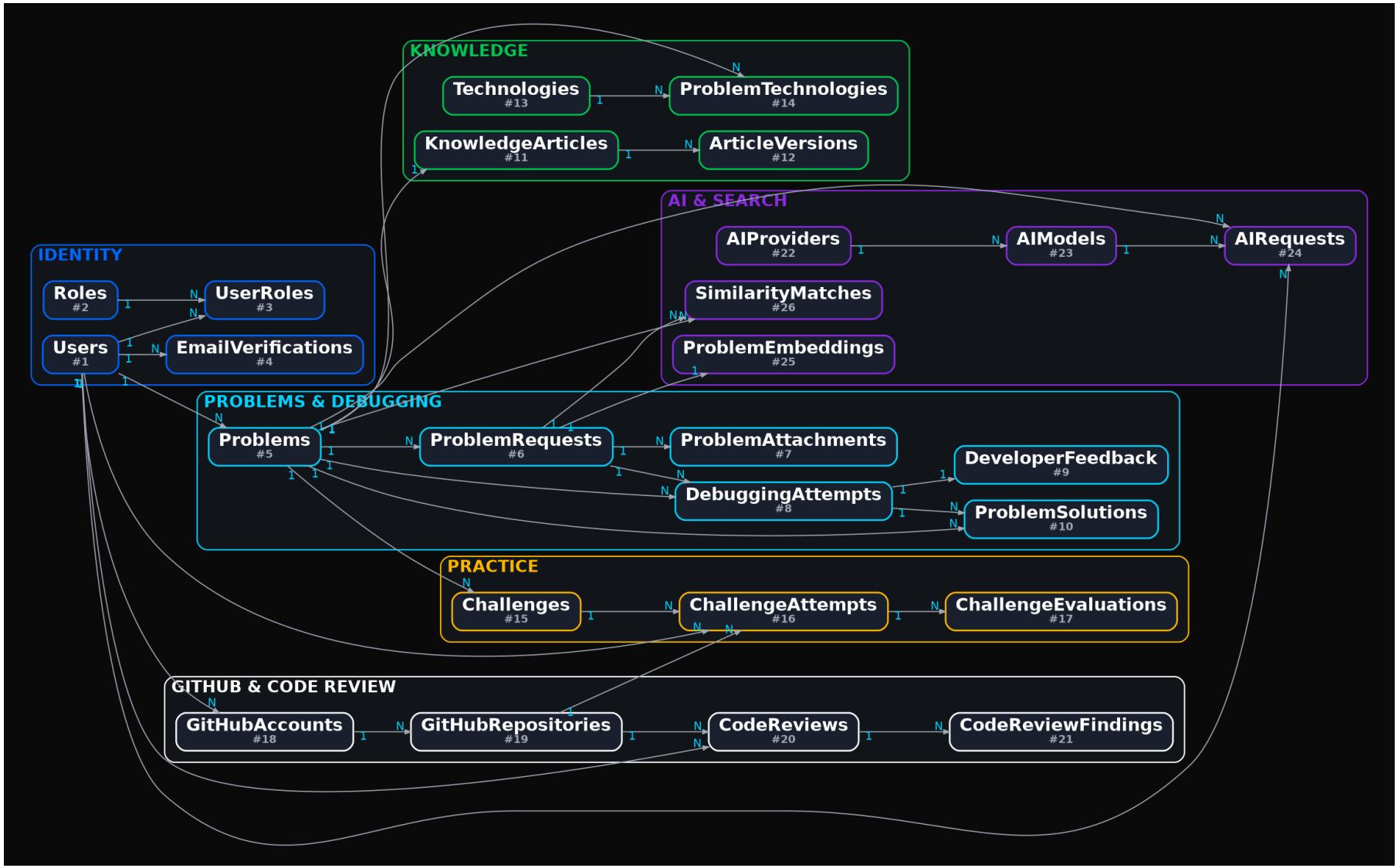


Figure 1. Core ERD -26 tables in six groups. Each edge reads 1 (parent) to N (child), or 1:1 where both ends are 1.

3. Table Documentation

Each table is documented with its purpose, columns, and relationships. Key markers: PK = primary key, FK = foreign key.

3.1 Users

PURPOSE

Stores platform users. The main application user is the Developer.

Columns:

Column	Data Type	Key	Description
UserId	INT	PK	Primary Key.
Name	NVARCHAR(150)		Developer display/name value.
Email	NVARCHAR(255)		User email.
PasswordHash	NVARCHAR(500)		Stored password hash.
CreatedAt	DATETIME2		Account creation time.
EmailVerifiedAt	DATETIME2 NULL		Timestamp indicating when the user's email was successfully verified.

Relationships:

- One User can have many Problems.
- One User can have many ChallengeAttempts.
- One User can have many GitHubAccounts.
- One User can have many CodeReviews.
- One User can have many AIRequests.
- One User can have many UserRoles.
- One User can have many EmailVerifications.

3.2 Roles

PURPOSE

Stores application roles.

Columns:

Column	Data Type	Key	Description
RoleId	INT	PK	Primary Key.
Name	NVARCHAR(50)		Role name.

Initial Roles:

- Developer
- Administrator

Relationships:

- One Role can be assigned to many UserRoles.

3.3 UserRoles

> **PURPOSE**
> Junction table connecting users and roles.

Columns:

Column	Data Type	Key	Description
`UserId`	INT	PK, FK	Primary Key + Foreign Key to Users.UserId.
`RoleId`	INT	PK, FK	Primary Key + Foreign Key to Roles.RoleId.

Key:

```
Composite Primary Key: (UserId, RoleId)
```

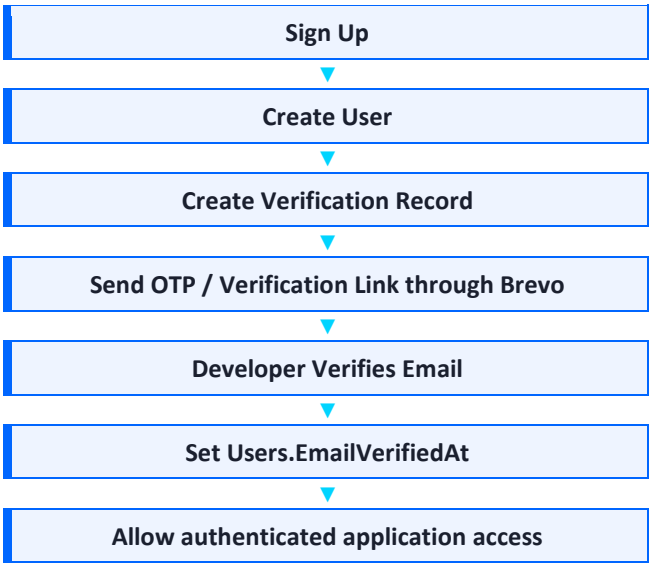
Relationship:

- Users 1:N UserRoles
- Roles 1:N UserRoles

3.4 EmailVerifications

> **PURPOSE**
> Stores email verification attempts for account activation.
>
> The verification process is used during Sign Up and is handled through Brevo as the project's email delivery service.

Main flow:



Columns:

Column	Data Type	Key	Description
`VerificationId`	INT	PK	Primary Key.

Column	Data Type	Key	Description
UserId	INT	FK	Foreign Key to Users.UserId.
CodeHash	NVARCHAR(500)		Hashed OTP verification code.
TokenHash	NVARCHAR(500) NULL		Hashed verification token when link-based verification is used.
Method	VARCHAR(20)		Verification method, for example: OTP, Link.
ExpiresAt	DATETIME2		Expiration time of the verification request.
Attempts	INT		Number of verification attempts.
VerifiedAt	DATETIME2 NULL		Time the verification was successfully completed.
CreatedAt	DATETIME2		Verification record creation time.

Relationships:

- Users 1:N EmailVerifications.

Security Rules:

- Never store the raw OTP in the database.
- Store the OTP as a secure hash.
- Never expose email provider credentials to the frontend.
- Expired verification records must not be accepted.
- Previous verification codes/tokens should be invalidated when a new verification request is issued.
- Resend attempts should be protected by cooldown/rate limiting.

Email Provider:

- Brevo

> **Brevo (external service)**
> Brevo is an external service and does not require a dedicated database table in the current Core ERD. The database stores verification state; Brevo is responsible for email delivery.

3.5 Problems

> PURPOSE
> Core business entity representing a developer's technical problem.

Columns:

Column	Data Type	Key	Description
ProblemId	INT	PK	Primary Key.
UserId	INT	FK	Foreign Key to Users.UserId.
ProblemCode	VARCHAR(20)		Public unique identifier such as DEV-000001.
Title	NVARCHAR(250)		Problem title.
Description	NVARCHAR(MAX)		Main problem description.

Column	Data Type	Key	Description
Status	VARCHAR(30)		Problem status, for example active/solved according to application rules.
CreatedAt	DATETIME2		Problem creation time.
SolvedAt	DATETIME2 NULL		Time the problem was confirmed as solved.

Relationships:

- Users 1:N Problems.
- Problems 1:N ProblemRequests.
- Problems 1:N DebuggingAttempts.
- Problems 1:N ProblemSolutions.
- Problems 1:1 KnowledgeArticles.
- Problems 1:N ProblemTechnologies.
- Problems 1:N Challenges.
- Problems 1:N AIRequests.
- Problems 1:N SimilarityMatches.

3.6 ProblemRequests

> **PURPOSE**
> Stores every request/input related to a problem.
>
> This is important because the same problem can have multiple requests while the developer continues the debugging loop.

Columns:

Column	Data Type	Key	Description
RequestId	INT	PK	Primary Key.
ProblemId	INT	FK	Foreign Key to Problems.ProblemId.
Content	NVARCHAR(MAX)		Main submitted content.
ErrorMessage	NVARCHAR(MAX) NULL		Error/exception message when supplied.
StackTrace	NVARCHAR(MAX) NULL		Stack trace when supplied.
TechnologyHint	NVARCHAR(200) NULL		Developer-provided technology/framework hint.
CreatedAt	DATETIME2		Request creation time.

Relationships:

- Problems 1:N ProblemRequests.
- ProblemRequests 1:N ProblemAttachments.
- ProblemRequests 1:N DebuggingAttempts.
- ProblemRequests 1:1 ProblemEmbeddings.
- ProblemRequests 1:N SimilarityMatches.

3.7 ProblemAttachments

> **PURPOSE**
> Stores attachments associated with a submitted problem request.

Columns:

Column	Data Type	Key	Description
AttachmentId	INT	PK	Primary Key.
RequestId	INT	FK	Foreign Key to ProblemRequests.RequestId.
FileType	VARCHAR(30)		Attachment type.
FileName	NVARCHAR(255)		Original/file display name.
StoragePath	NVARCHAR(500)		Storage reference/path.

Relationship:

- ProblemRequests 1:N ProblemAttachments.

3.8 DebuggingAttempts

> **PURPOSE**
> Stores each AI debugging attempt for a problem.
>
> A problem can contain multiple attempts because the developer may select “Still Have a Problem”.

Columns:

Column	Data Type	Key	Description
AttemptId	INT	PK	Primary Key.
ProblemId	INT	FK	Foreign Key to Problems.ProblemId.
RequestId	INT	FK	Foreign Key to ProblemRequests.RequestId.
AttemptNumber	INT		Attempt sequence number.
RootCause	NVARCHAR(MAX)		AI-generated root cause explanation.
SolutionText	NVARCHAR(MAX)		Proposed solution.
WhyItWorks	NVARCHAR(MAX)		Explanation of why the solution should work.
CreatedAt	DATETIME2		Attempt creation time.

Relationships:

- Problems 1:N DebuggingAttempts.
- ProblemRequests 1:N DebuggingAttempts.
- DebuggingAttempts 1:1 DeveloperFeedback.
- DebuggingAttempts 1:N ProblemSolutions.

3.9 DeveloperFeedback

> **PURPOSE**
> Stores the developer's result after a debugging attempt.

Main result values:

- Solved
- Still Have a Problem

Columns:

Column	Data Type	Key	Description
FeedbackId	INT	PK	Primary Key.
AttemptId	INT	FK	Foreign Key to DebuggingAttempts.AttemptId.
Result	VARCHAR(30)		Feedback result.
Comment	NVARCHAR(MAX) NULL		Additional explanation from the developer.
CreatedAt	DATETIME2		Feedback creation time.

Relationship:

- DebuggingAttempts 1:1 DeveloperFeedback.

3.10 ProblemSolutions

> **PURPOSE**
> Stores reusable successful solution records for a problem.

This table is important for:

- Solution reuse.
- Similar problem handling.
- Token optimization.
- Tracking successful variants.
- Selecting a preferred solution.

Columns:

Column	Data Type	Key	Description
SolutionId	INT	PK	Primary Key.
ProblemId	INT	FK	Foreign Key to Problems.ProblemId.
SourceAttemptId	INT	FK	Foreign Key to DebuggingAttempts.AttemptId.
SolutionText	NVARCHAR(MAX)		Stored solution content.
SuccessCount	INT		Number of successful uses.
LastSuccessfulAt	DATETIME2		Last known successful use.
IsPreferred	BIT		Indicates whether the solution is the currently preferred solution.

Relationships:

- Problems 1:N ProblemSolutions.
- DebuggingAttempts 1:N ProblemSolutions.

3.11 KnowledgeArticles

> **PURPOSE**
> Represents the public educational article generated from a solved problem.

Columns:

Column	Data Type	Key	Description
ArticleId	INT	PK	Primary Key.
ProblemId	INT	FK	Foreign Key to Problems.ProblemId.
Title	NVARCHAR(250)		Article title.
CurrentVersionId	INT NULL		Reference to the current ArticleVersions record.
Status	VARCHAR(30)		Article status.
UpdatedAt	DATETIME2		Last article update time.

Relationships:

- Problems 1:1 KnowledgeArticles.
- KnowledgeArticles 1:N ArticleVersions.

3.12 ArticleVersions

> **PURPOSE**
> Stores version history for educational articles.
>
> This allows the article to improve as more successful solutions are collected.

Columns:

Column	Data Type	Key	Description
VersionId	INT	PK	Primary Key.
ArticleId	INT	FK	Foreign Key to KnowledgeArticles.ArticleId.
VersionNumber	INT		Article version sequence.
ArticleContent	NVARCHAR(MAX)		Full article content.
UpdateReason	NVARCHAR(500)		Reason for generating/updating the version.
CreatedAt	DATETIME2		Version creation time.

Relationship:

- KnowledgeArticles 1:N ArticleVersions.

3.13 Technologies

> **PURPOSE**
> Stores technologies/frameworks associated with problems.

Examples:

- ASP.NET Core
- SQL Server
- React
- Angular
- Python

Columns:

Column	Data Type	Key	Description
`TechnologyId`	`INT`	`PK`	Primary Key.
`Name`	`NVARCHAR(100)`		Technology name.

Relationship:

- Technologies 1:N ProblemTechnologies.

3.14 ProblemTechnologies

> **PURPOSE**
> Junction table connecting Problems with Technologies.

Columns:

Column	Data Type	Key	Description
`ProblemId`	`INT`	`PK, FK`	Primary Key + Foreign Key to Problems.ProblemId.
`TechnologyId`	`INT`	`PK, FK`	Primary Key + Foreign Key to Technologies.TechnologyId.

Key:

```
Composite Primary Key: (ProblemId, TechnologyId)
```

Relationships:

- Problems 1:N ProblemTechnologies.
- Technologies 1:N ProblemTechnologies.

4. Practice / Challenge Tables

4.1 Challenges

> **PURPOSE**
> Stores coding challenges generated from solved problems.

Columns:

Column	Data Type	Key	Description
ChallengeId	INT	PK	Primary Key.
ProblemId	INT	FK	Foreign Key to Problems.ProblemId.
Title	NVARCHAR(250)		Challenge title.
Description	NVARCHAR(MAX)		Challenge statement.
Difficulty	VARCHAR(30)		Challenge difficulty.
Technology	NVARCHAR(100)		Main technology associated with the challenge.
CreatedAt	DATETIME2		Creation time.

Relationship:

- Problems 1:N Challenges.

4.2 ChallengeAttempts

> **PURPOSE**
> Stores each developer's attempt to solve a challenge.

Columns:

Column	Data Type	Key	Description
AttemptId	INT	PK	Primary Key.
ChallengeId	INT	FK	Foreign Key to Challenges.ChallengeId.
UserId	INT	FK	Foreign Key to Users.UserId.
GitHubRepositoryId	INT NULL	FK	Optional Foreign Key to GitHubRepositories.GitHubRepositoryId.
Status	VARCHAR(30)		Attempt status.
StartedAt	DATETIME2		Attempt start time.
CompletedAt	DATETIME2 NULL		Completion time.

Relationships:

- Users 1:N ChallengeAttempts.
- Challenges 1:N ChallengeAttempts.
- GitHubRepositories 1:N ChallengeAttempts.
- ChallengeAttempts 1:N ChallengeEvaluations.

4.3 ChallengeEvaluations

> **PURPOSE**
> Stores the AI evaluation of a challenge implementation.

Columns:

Column	Data Type	Key	Description
EvaluationId	INT	PK	Primary Key.
AttemptId	INT	FK	Foreign Key to ChallengeAttempts.AttemptId.
FunctionalScore	DECIMAL(5,2)		Functional correctness score.
CodeQualityScore	DECIMAL(5,2)		Code quality score.
SecurityScore	DECIMAL(5,2)		Security score.
PerformanceScore	DECIMAL(5,2)		Performance score.
Feedback	NVARCHAR(MAX)		Detailed evaluation feedback.
CreatedAt	DATETIME2		Evaluation creation time.

Relationship:

- ChallengeAttempts 1:N ChallengeEvaluations.

5. GitHub & Code Review Tables

5.1 GitHubAccounts

PURPOSE

Stores a developer's connected GitHub account.

Columns:

Column	Data Type	Key	Description
GitHubAccountId	INT	PK	Primary Key.
UserId	INT	FK	Foreign Key to Users.UserId.
GitHubUserName	NVARCHAR(100)		GitHub username.
AccessTokenEncrypted	NVARCHAR(2000)		Encrypted GitHub access credential/reference.
ConnectedAt	DATETIME2		Connection time.

Relationship:

- Users 1:N GitHubAccounts.

Security Note

The access token must never be exposed to the frontend and must be protected using secure secret handling.

5.2 GitHubRepositories

PURPOSE

Stores repositories connected through a GitHub account.

Columns:

Column	Data Type	Key	Description
GitHubRepositoryId	INT	PK	Internal Primary Key.
GitHubAccountId	INT	FK	Foreign Key to GitHubAccounts.GitHubAccountId.
ExternalRepositoryId	VARCHAR(100)		GitHub repository identifier.
RepositoryName	NVARCHAR(200)		Repository name.
OwnerName	NVARCHAR(150)		Repository owner.
IsPrivate	BIT		Indicates private/public repository visibility.

Relationships:

- GitHubAccounts 1:N GitHubRepositories.
- GitHubRepositories 1:N ChallengeAttempts.
- GitHubRepositories 1:N CodeReviews.

5.3 CodeReviews

> **PURPOSE**
> Stores an AI code review operation.

Columns:

Column	Data Type	Key	Description
CodeReviewId	INT	PK	Primary Key.
GitHubRepositoryId	INT	FK	Foreign Key to GitHubRepositories.GitHubRepositoryId.
UserId	INT	FK	Foreign Key to Users.UserId.
BranchName	NVARCHAR(255) NULL		Selected branch.
PullRequestNumber	INT NULL		Optional pull request number.
Status	VARCHAR(30)		Review status.
CreatedAt	DATETIME2		Review creation time.

Relationships:

- Users 1:N CodeReviews.
- GitHubRepositories 1:N CodeReviews.
- CodeReviews 1:N CodeReviewFindings.

5.4 CodeReviewFindings

> **PURPOSE**
> Stores individual findings returned by an AI code review.

Columns:

Column	Data Type	Key	Description
FindingId	INT	PK	Primary Key.
CodeReviewId	INT	FK	Foreign Key to CodeReviews.CodeReviewId.
FilePath	NVARCHAR(500)		File containing the finding.
Severity	VARCHAR(30)		Finding severity.
Category	VARCHAR(50)		Finding category.
Problem	NVARCHAR(MAX)		Description of the issue.
Suggestion	NVARCHAR(MAX)		Suggested improvement.

Relationship:

- CodeReviews 1:N CodeReviewFindings.

6. AI & Search Tables

6.1 AIProviders

PURPOSE

Stores AI/LLM provider configuration.

The platform uses an AI Gateway so application modules do not depend directly on one provider.

Columns:

Column	Data Type	Key	Description
ProviderId	INT	PK	Primary Key.
Name	NVARCHAR(100)		Provider name.
BaseUrl	NVARCHAR(500)		Provider API endpoint/base URL.
Priority	INT		Provider routing priority.
IsActive	BIT		Provider active status.

Relationship:

- AIProviders 1:N AIModels.

6.2 AIModels

PURPOSE

Stores models belonging to an AI provider and their operational usage state.

Columns:

Column	Data Type	Key	Description
ModelId	INT	PK	Primary Key.

Column	Data Type	Key	Description
ProviderId	INT	FK	Foreign Key to AIProviders.ProviderId.
ModelName	NVARCHAR(150)		Model name.
RequestsUsed	INT		Tracked request count.
TokensUsed	BIGINT		Tracked token usage when available.
IsAvailable	BIT		Current availability state.

Relationship:

- AIProviders 1:N AIModels.
- AIModels 1:N AIRequests.

6.3 AIRequests

> PURPOSE
> Stores AI processing requests and responses.

This supports:

- Debugging.
- Article generation.
- Challenge generation.
- Challenge evaluation.
- Code review.
- Repository analysis.
- Solution synthesis.
- Usage tracking.

Columns:

Column	Data Type	Key	Description
AIRequestId	BIGINT	PK	Primary Key.
ModelId	INT	FK	Foreign Key to AIModels.ModelId.
UserId	INT NULL	FK	Optional Foreign Key to Users.UserId.
ProblemId	INT NULL	FK	Optional Foreign Key to Problems.ProblemId.
OperationType	VARCHAR(50)		AI operation type.
RequestText	NVARCHAR(MAX)		Request/context sent to the AI service.
ResponseText	NVARCHAR(MAX) NULL		AI response.
TokensUsed	INT NULL		Token usage when available.
Status	VARCHAR(30)		Request status.
CreatedAt	DATETIME2		Request creation time.

Relationships:

- AIModels 1:N AIRequests.
- Users 1:N AIRequests.
- Problems 1:N AIRequests.

> **Security Note**
> Provider API keys must not be stored in this table.

6.4 ProblemEmbeddings

> PURPOSE
> Stores an embedding representation associated with a problem request to support semantic similarity/search.

Columns:

Column	Data Type	Key	Description
EmbeddingId	BIGINT	PK	Primary Key.
RequestId	INT	FK	Foreign Key to ProblemRequests.RequestId.
EmbeddingData	VARBINARY(MAX)		Stored embedding data.
CreatedAt	DATETIME2		Embedding creation time.

Relationship:

- ProblemRequests 1:1 ProblemEmbeddings.

> **Implementation Note**
> The exact embedding model, vector dimension, and similarity engine are not fixed by the current SRS/ERD and should be decided during implementation.

6.5 SimilarityMatches

> PURPOSE
> Stores similarity-search results connecting a new request with a historical problem.
>
> This supports token optimization and reuse of successful solutions.

Columns:

Column	Data Type	Key	Description
MatchId	BIGINT	PK	Primary Key.
RequestId	INT	FK	Foreign Key to ProblemRequests.RequestId.
MatchedProblemId	INT	FK	Foreign Key to Problems.ProblemId.
SimilarityScore	DECIMAL(6,5)		Similarity result.
CreatedAt	DATETIME2		Match creation time.

Relationships:

- ProblemRequests 1:N SimilarityMatches.
- Problems 1:N SimilarityMatches.

7. Relationship Summary

Users	1:N Problems 1:N UserRoles 1:N EmailVerifications 1:N ChallengeAttempts 1:N GitHubAccounts 1:N CodeReviews 1:N AIRequests
Roles	1:N UserRoles
Problems	1:N ProblemRequests 1:N DebuggingAttempts 1:N ProblemSolutions 1:1 KnowledgeArticles 1:N ProblemTechnologies 1:N Challenges 1:N AIRequests 1:N SimilarityMatches
ProblemRequests	1:N ProblemAttachments 1:N DebuggingAttempts 1:1 ProblemEmbeddings 1:N SimilarityMatches
DebuggingAttempts	1:1 DeveloperFeedback 1:N ProblemSolutions
KnowledgeArticles	1:N ArticleVersions
Technologies	1:N ProblemTechnologies
Challenges	1:N ChallengeAttempts
ChallengeAttempts	1:N ChallengeEvaluations
GitHubAccounts	1:N GitHubRepositories
GitHubRepositories	1:N ChallengeAttempts 1:N CodeReviews
CodeReviews	1:N CodeReviewFindings
AIProviders	1:N AIModels
AIModels	1:N AIRequests

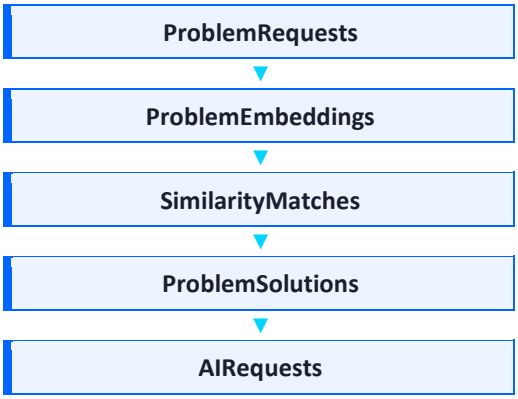
1:N = one-to-many 1:1 = one-to-one

8. Core Business Data Flow

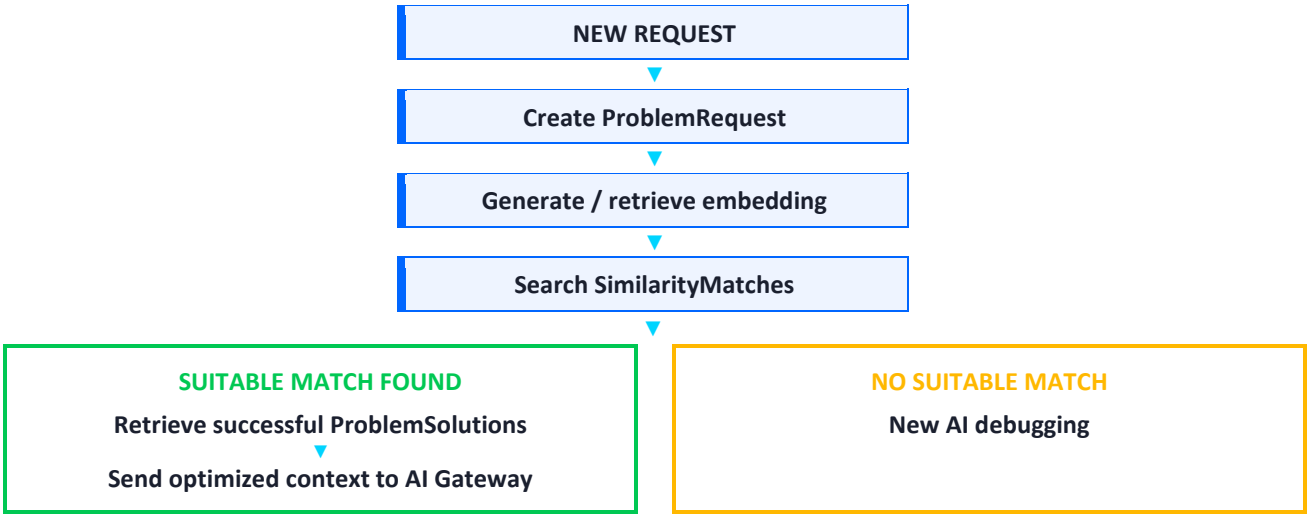
1	Developer registers an account.
2	The system creates an email verification record and sends an OTP/verification link through Brevo.
3	After successful verification, the system sets Users.EmailVerifiedAt and allows authenticated access.
4	Developer creates a Problem.
5	Problem can receive multiple ProblemRequests.
6	Each request can include attachments, errors, stack traces and context.
7	The system searches for similar historical problems.
8	Similarity results are stored.
9	The AI Gateway sends optimized context to an available model.
10	Each debugging attempt is stored.
11	Developer gives feedback: → **Solved** → **Still Have a Problem**
12	If unsolved, another request/attempt is stored and the debugging loop continues.
13	When solved, the successful solution is stored in ProblemSolutions.
14	The system generates or updates KnowledgeArticles.
15	Article history is stored in ArticleVersions.
16	A Challenge can be generated from the problem.
17	A developer can start a ChallengeAttempt.
18	A GitHub repository can be associated with the attempt.
19	The submitted implementation can receive a ChallengeEvaluation.
20	Similar future requests can reuse successful solutions.
21	Multiple successful solution variants can later be synthesized into improved reusable knowledge.

9. Token Optimization Data Model

The token-saving workflow is primarily supported by:



Recommended logical flow:

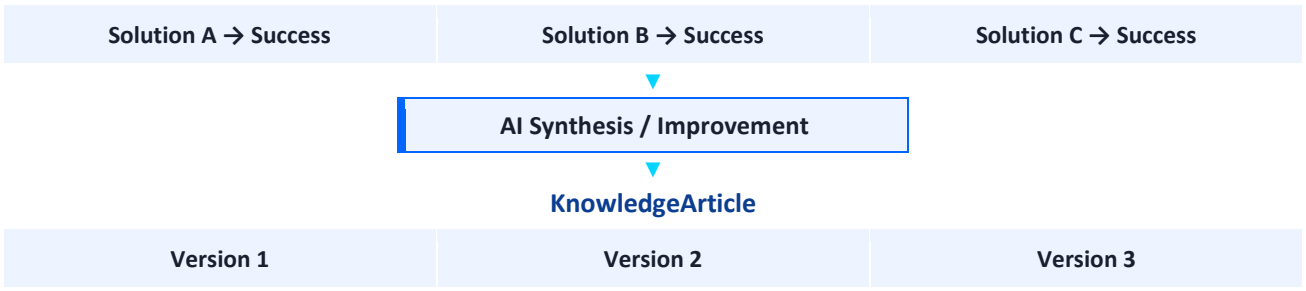


> Every successful new adaptation should be stored so it can be reused in future requests.

10. Article Evolution Data Model

A problem may accumulate multiple successful solutions.

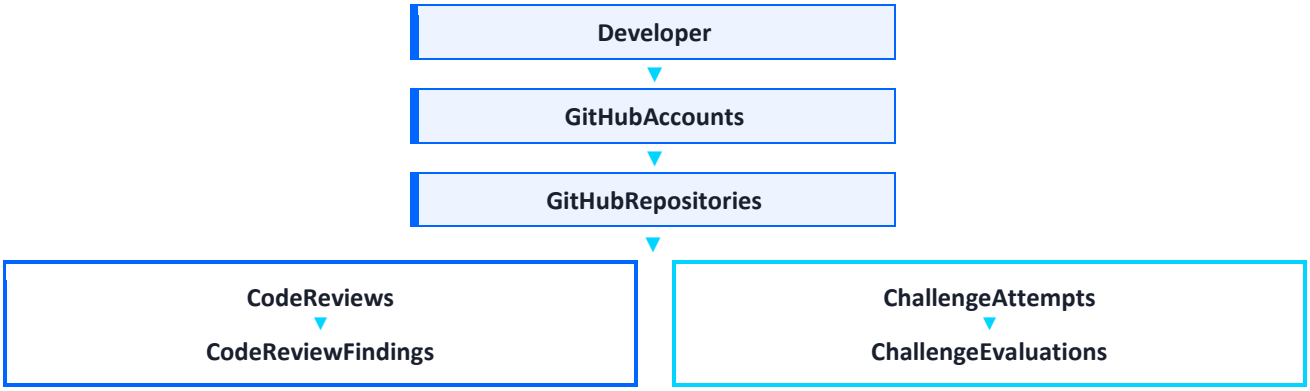
Example -Problem X:



The previous article versions are preserved.

The current article points to the current version through `CurrentVersionId`.

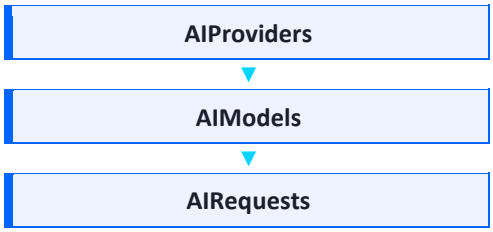
11. GitHub Data Flow



GitHub access must use the minimum permissions required by each feature.

Private repository content must not become public knowledge content unless explicitly and appropriately handled by the application rules.

12. AI Data Flow



AIRequest can optionally belong to:

- A User
- A Problem

AI operation examples:

Debugging	**Root Cause Analysis**	**Article Generation**	**Challenge Generation**
Challenge Evaluation	**Code Review**	**Repository Analysis**	**Solution Synthesis**

The AI Gateway is responsible for selecting providers/models, handling errors, retrying, failing over, and tracking usage.

13. Important Constraints

13.1 Security

- GitHub access credentials must be protected.
- LLM provider keys must never be exposed to the frontend.
- Sensitive repository information must not be exposed publicly.
- The database should not store unnecessary secrets.

13.2 Privacy

- Private repository information must remain restricted.
- Problem data should be handled according to project privacy rules.
- Sensitive credentials should not be placed in AI prompts.

13.3 History

- Previous debugging attempts must not be deleted merely because a later attempt succeeds.
- Previous article versions should be preserved.

13.4 Knowledge Quality

- Successful usage is evidence, but popularity alone must not be treated as proof of correctness.
- Knowledge improvement should consider successful outcomes and review.

14. Indexing / Constraints to Define During Implementation

The Core ERD defines the key relationships, but final SQL Server indexing strategy should be finalized during implementation based on actual query patterns.

Priority candidates for uniqueness/indexing

- `Users.Email`
- `Problems.ProblemCode`
- `Roles.Name`
- `Technologies.Name`
- `GitHubRepositories.ExternalRepositoryId`
- `AIModels.ProviderId`
- `SimilarityMatches.RequestId`
- `SimilarityMatches.MatchedProblemId`

Likely search-heavy columns

- `Problems.ProblemCode`
- `Problems.Title`
- `Problems.Status`
- `ProblemRequests.ErrorMessage`
- `ProblemRequests.TechnologyHint`
- `Technologies.Name`
- `Challenges.Title`
- `GitHubRepositories.RepositoryName`

Exact indexes and uniqueness rules should be confirmed in the database implementation before migrations are finalized.

15. Transactional Areas

Operations that update multiple related records should be handled transactionally where appropriate.

Examples:

Problem solved

- Update Problems.Status / SolvedAt
- Store successful ProblemSolution
- Update statistics/state
- Create or update KnowledgeArticle
- Create ArticleVersion

Challenge completion

- Update ChallengeAttempt
- Store ChallengeEvaluation
- Update any related challenge state

AI operation

- Store AIRequest metadata/result
- Update usage tracking
- Update provider/model availability when required

16. Core Tables vs Optional Supporting Tables

Included in the Core ERD

- 26 tables documented in this file.

Not included in the Core ERD currently

- Notifications
- DiscordEvents
- DiscordUsers
- AuditLogs
- Additional operational/support tables that may be introduced later

Reason

The current database model focuses on the central product workflow and keeps the ERD understandable.

These supporting areas can be added later without changing the central Problem → Solution → Knowledge → Challenge → GitHub → AI model.

17. Database Implementation Order

Recommended implementation order:

1	Users	IDENTITY
2	Roles	
3	UserRoles	
4	EmailVerifications	
5	Problems	PROBLEMS & DEBUGGING
6	ProblemRequests	
7	ProblemAttachments	
8	DebuggingAttempts	
9	DeveloperFeedback	
10	ProblemSolutions	
11	Technologies	KNOWLEDGE
12	ProblemTechnologies	
13	KnowledgeArticles	
14	ArticleVersions	
15	Challenges	PRACTICE
16	ChallengeAttempts	
17	ChallengeEvaluations	
18	GitHubAccounts	GITHUB & CODE REVIEW
19	GitHubRepositories	
20	CodeReviews	
21	CodeReviewFindings	
22	AIProviders	AI
23	AIModels	
24	AIRequests	
25	ProblemEmbeddings	SEARCH
26	SimilarityMatches	

This order follows the main dependency structure of the current Core ERD.

18. Recent Approved Changes

The database documentation was updated to reflect the latest approved project decisions:

1. Email verification is now part of the official authentication workflow.
2. Brevo is the selected email delivery service.
3. Users now store EmailVerifiedAt.

4. EmailVerifications was added as a Core ERD table.
5. OTPs/tokens are represented as hashes, not raw values.
6. GitHub access tokens are represented as encrypted storage.
7. The Core ERD now contains 26 tables.
8. The database implementation order was updated accordingly.

> The email provider itself remains an external integration and is intentionally not modeled as a database table.

19. Final Database Purpose

The database is not only a storage layer for chat messages.

It is designed to preserve the complete lifecycle of a real developer problem:



The central database goal is therefore:

> ***"Store real developer problems and proven solutions in a reusable structure that can improve future debugging, educational content, practical challenges, and AI efficiency."***

END OF DATABASE DOCUMENTATION